

Atividade de AEDS III - Tabelas hash + Huffman coding

Objetivo

Construir um compactador capaz de ler um arquivo de com leituras de um sensor (com medidas de temperatura, umidade, índice de raios UV e qualidade do ar), calcular as frequências de cada medida usando **tabela hash**, gerar a **árvore de Huffman** correspondente, comprimir os dados e depois restaurá-lo, garantindo que os dados sejam consistentes.

Instruções

1. Leitura do arquivo com dados do sensor
2. Criação de **tabela hash** que calcula a frequência (número de ocorrências) de cada pacote (conjunto de medidas).
3. Criação de **árvore de Huffman** para gerar correspondentes dos dados com compressão.
4. Geração de arquivos de saída com dados comprimidos em binário e tabela de equivalência.
5. Implementação do processo inverso - gerar o arquivo de dados do sensor a partir dos dados compactados.

Detalhes da implementação

1 - Especificação dos arquivos

Arquivo	Papel	Formato exato (! obrigatório)
<code>sensor.log</code>	Entrada principal	Texto ASCII. Cada letra é uma representação em ASCII de um hexadecimal de 4 bytes formando uma string com 8 caracteres (cada byte - par de letras - corresponde a uma medida: temp, humidade, UV, ar). Ex.: <code>7F3A0042</code>
<code>sensor.tbl</code>	Saída (intermediária)	Arquivo binário (não é ASCII) contendo dados ordenados por frequência e o código de Huffman que o representa, ajustado para um ou mais bytes quando necessário. Ex: <code>7F3A0042</code> (4 bytes) equivale ao código 11 (1 byte):

01111111 00111010 00000000 01000010
00000011

<code>sensor.hzp</code>	Saída (final)	Arquivo binário (não é ASCII): cabeçalho + bits compactados
<code>stats.txt</code>	Saída (relatório)	Texto com métricas

2 - Regras de implementação (validadas em correção)

1. Implementação da tabela hash

- Pode ser implementada da forma preferida, mas precisa ser implementada manualmente (sem o uso de bibliotecas).
- Verifique os códigos disponíveis no material da disciplina.

2. Árvore de Huffman

- Construa a fila de prioridade mínima **usando um min-heap** conforme exemplo de código disponível no material.

3. Uso de bibliotecas

- É **proibido usar bibliotecas** prontas para as estruturas de dados: `unordered_map`, `std::map`, `dict`, `HashMap`, `PriorityQueue`, `heapq`, etc.
- É permitido apenas bibliotecas padrão da linguagem, manipulação de bits e containers básicos (vetor/array e `struct/class`).

4. Linguagem:

- Qualquer linguagem poderá ser utilizada desde que as estruturas de dados sejam **implementadas manualmente**.

3 - Interface de linha de comando

- Seu código deverá produzir um binário chamado `hzip`, que deverá aceitar dois tipos de entrada:

```
$ ./hzip c
```

```
$ ./hzip d
```

- Modo **c** deve fazer a compressão - ler o arquivo de entrada **sensor.log** e gerar as saídas: **sensor.tbl**, **sensor.hzp**, **stats.txt**.
- Modo **d** deve fazer a descompressão - ler os arquivos de entrada **sensor.tbl** e **sensor.hzp** para produzir **output.log** idêntico byte a byte ao **sensor.log** original.

4 - Formato de **sensor.hzp**

Cabeçalho	Número linhas arquivo fonte	Dados compactados
4 bytes ASCII: HZIP	4 bytes (32 bit int): tamanho	Dados binários

1. **Cabeçalho:** **HZIP** para validar entrada de descompressão.
2. **Tamanho:** **número de linhas** do arquivo fonte.
3. **Dados:** sequência de **códigos Huffman** concatenados em dados binários.

5 - Requisitos funcionais mínimos

- Arquivo **sensor.tbl** **sem códigos duplicados** e com **bits dos símbolos**.
- Arquivo **sensor.hzp** decodifica para uma cópia **perfeita** de **sensor.log**.
- Arquivo **sensor.hzp** deve ser **estritamente menor** que **sensor.log**. **Para garantir isso a gravação dos dados deve ser feita usando binários e não ASCII.**

6 - **stats.txt** (modelo)

```

Pacotes distintos .....: 1056
Total de pacotes .....: 250000
Tamanho original (bytes) ...: 2 000 000
Tamanho compactado (bytes)..: 1 034 217
Razão de compressão (%) ....: 48.3
Tempo hash (ms) .....: 713
Tempo Huffman (ms) .....: 68
Tempo compressão (ms) .....: 802
Tempo descompressão (ms) ...: 789

```

Use `clock_gettime(CLOCK_MONOTONIC, ...)` ou qualquer outra função de medição do tempo.

7 - Resultados esperados (avaliação)

- Desenvolvimento de uma aplicação completa e de uso prático.
- Implementação na mão de todas as estruturas de dados e algoritmos necessários.
- Além dos conhecimentos dos algoritmos, outros requisitos devem surgir durante a execução do trabalho (**manipulação de bits, conversão de dados hexadecimal para binário e vice-versa, escrita e leitura em arquivo e cálculo do tempo das operações, entre outros**). Parte do trabalho consiste em aprender a lidar com os recursos disponíveis para gerar uma solução para o problema. A pesquisa de como implementar essas funcionalidades é crucial para completar o trabalho.
- **Qualquer programa deverá ser capaz de descomprimir os dados de outra implementação.**
- Comparação do desempenho (temporal) das várias implementações realizadas.

Exemplo de entrada e saída

1 - Arquivo de entrada (`sensor.log`) - em ASCII

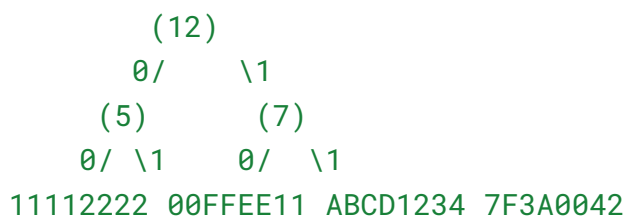
```
7F3A0042
7F3A0042
ABCD1234
7F3A0042
00FFEE11
11112222
ABCD1234
00FFEE11
00FFEE11
ABCD1234
11112222
7F3A0042
```

12 linhas; cada linha é um pacote hexadecimal de 4 bytes e cada byte representa uma medida: temperatura, humidade, UV e qualidade do ar).

2 - Leitura + tabela hash

Pacote	Frequência
7F3A0042	4
ABCD1234	3
00FFEE11	3
11112222	2

3 - Árvore de Huffman gerada



4 - Códigos Huffman resultantes (`sensor.tbl`) em ordem de frequência

```

01111111001110100000000001000010 00000011
10101011110011010001001000110100 00000010
0000000011111111110111000010001 00000001
00010001000100010010001000100010 00000000

```

5 - Compressão

```
11 11 10 11 01 00 10 01 01 10 00 11
```

1. Leitura linear – percorre `sensor.log` linha a linha, exatamente na ordem em que os pacotes aparecem.
2. Bitstream resultante: `111110110100100101100011` (24 bits).
3. Agrupado em bytes: `FB 49 63` (hex).

6 - Formato do `sensor.hzp` (binário)

Posição	Conteúdo	Hexadecimal
0x00	"HZIP" (cabeçalho)	48 5A 49 50
0x04	nPacotes = 12 (uint32_t)	00 00 00 0C
0x08	Dados (bits)	FB 49 63

Tamanho final: 11 bytes.